Anomaly Detection — Credit Card Fraud

This notebook implements an **Isolation Forest** anomaly detection pipeline against the Credit Card Fraud Detection dataset registered in Azure ML Assets. It runs identically from **Azure AI Machine Learning Studio** → **Authoring** → **Notebooks** or from a local Python environment backed by `requirements.txt`.

Business Context

Credit card fraud imposes measurable costs at two distinct failure modes that cut in opposite directions. Getting the balance wrong in either direction has direct financial and reputational consequences.

The Cost of Getting It Wrong — In Both Directions

Failure mode	What happened	Who bears the cost	Estimated cost per event
False Negative (missed fraud)	A fraudulent transaction was not flagged	Issuing bank (chargeback liability), cardholder (stress, time)	**€122 avg. fraud loss** (ECB 2022); plus chargeback processing fees of €15–€40 per case
False Positive (flagged legitimate tx)	A legitimate transaction was blocked or queued for review	Cardholder (declined purchase, friction), merchant (lost sale), bank (customer service cost)	**€5–€25 per analyst review**; up to **3–5% customer churn** after a false decline

Why this asymmetry matters for model design: At 29% fraud-class precision and 28% recall (see Step 5), this model flags roughly **3.4 legitimate transactions for every genuine fraud case it catches**, while missing 72% of fraud entirely. Deploying this as a hard block would generate excessive friction. The right deployment pattern is a **risk tiering system** — see the Deployment Recommendations section at the end of this notebook.

Stakeholder Map

Different stakeholders have fundamentally different tolerances for each failure mode. All model changes and deployment decisions should be communicated through this lens:

Stakeholder	Primary concern	Acceptable false positive rate	Acceptable false negative rate
Risk & Compliance	Regulatory exposure (PSD2, GDPR, Reg E); total fraud loss	Low — prefers catching more fraud	Very low — missed fraud = direct liability

Stakeholder	Primary concern	Acceptable false positive rate	Acceptable false negative rate
Product / UX	Customer friction, checkout conversion rate	Very low — blocked legitimate purchases damage NPS	Moderate — occasional missed fraud is tolerable if rare
Finance	Net cost of fraud programme vs. prevented losses	Moderate	Low — unchecked fraud erodes margin
Customer Service	Volume of fraud disputes and false-decline complaints	Very low — each FP generates a support ticket	Moderate
Engineering / MLOps	Model latency, infrastructure cost, retraining cadence	Moderate	Moderate

---

Prerequisites

Requirement	Details
Azure subscription & workspace	Subscription docs · Workspace docs
Registered dataset	The Kaggle `creditcardfraud` dataset registered in **Assets** → **Data** under the name `creditcard_fraud`
`config.json`	Workspace config file co-located with this notebook (download from ml.azure.com → workspace name → top-right menu)
Python environment	Activate the venv from the course repo — see `requirements.txt`

Pipeline Overview

```
┌──────────────────────────────────────────────────────────────────────┐
│                        Azure AI Machine Learning                     │
│                                                                      │
│   ┌──────────────┐    ┌──────────────────┐    ┌──────────────────┐  │
│   │  Workspace   │ ──▶│ Assets → Data    │ ──▶│ Authoring →      │  │
│   │ from_config  │    │ (creditcard_     │    │ Notebooks        │  │
│   │  (SDK v1)    │    │  fraud dataset)  │    │ (this notebook)  │  │
│   └──────────────┘    └──────────────────┘    └──────────────────┘  │
└──────────────────────────────────────────────────────────────────────┘
                                                  │ pandas DataFrame
                                                  ▼
                  ┌──────────────────────────────────────────────────┐
                  │                Python (local / compute)          │
                  │                                                  │
                  │  Standardise Amount ──▶ IsolationForest fit      │
                  │                                  │               │
                  │                                  ▼               │
                  │                       ┌──────────────────┐       │
                  │                       │ Training Script  │       │
                  │                       │ (regression model)       │
                  │                       └──────────────────┘       │
                  │                                  │               │
                  │                  ┌───────────────┘               │
                  │                  ▼                               │
                  │  classification_report ──▶ SHAP beeswarm         │
                  │                  │                               │
                  │                  ▼                               │
                  │  joblib.dump ──▶ Model.register (Azure ML)       │
                  └──────────────────────────────────────────────────┘
```

Step 1: Import Dependencies

All packages ship with the virtual environment defined in `requirements.txt`. The `azureml-core` SDK v1 is used for workspace auth and dataset retrieval; `scikit-learn` handles model training and evaluation; `shap` provides model explainability.

In [ ]:
```
from azureml.core import Workspace, Dataset          # azureml-core: workspace auth
import pandas as pd                                  # pandas: tabular data manipula
from sklearn.ensemble import IsolationForest         # scikit-learn: unsupervised an
from sklearn.metrics import classification_report    # scikit-learn: per-class preci
from azureml.core.model import Model                 # azureml-core: model registrat
```

Step 2: Authenticate and Load Dataset

`Workspace.from_config()` reads `config.json` from the current working directory (or a path you specify). In Azure ML Studio Notebooks the config is injected automatically; locally you must download and place it alongside this file.

> **Data transfer note:** `to_pandas_dataframe()` downloads ~150 MiB from Blob storage. Budget ~4 minutes on a local connection; near-instant on an Azure compute instance in the same region.

Diagnosing common workspace warnings

The following messages are informational — they do **not** indicate a failure:

Message	Cause	Action
`Falling back to use azure cli login credentials`	Using `az login` auth	Fine for dev; use `ServicePrincipalAuthentication` in CI/CD
`{'infer_column_types': 'False', 'activity': 'to_pandas_dataframe'}`	SDK telemetry during DataFrame conversion	Ignore
`Timeout was exceeded in force_flush()`	Background telemetry flush timed out	Ignore — no effect on data
`Overriding of current TracerProvider … is not allowed`	Duplicate OTel initialisation	Ignore
`Attempting to instrument while already instrumented`	SDK diagnostics already attached	Ignore

`config.json` not found in Azure ML Studio Notebooks

If you see `UserErrorException: The workspace configuration file config.json could not be found`, run the diagnostic cell below to confirm your working directory, then upload `config.json` alongside this notebook.

In [ ]:
```
# Diagnostic — run this if Workspace.from_config() raises UserErrorException.
# Confirms current directory and lists files so you can verify config.json is prese
import os
print("Working directory:", os.getcwd())
print("Directory contents:", os.listdir())
```

In [ ]:
```
# Set path=None when running inside Azure ML Studio Notebooks (config.json is auto-
# Set path explicitly when running locally, e.g. path='~/config.json'
path = None
# path = 'Users/<your-username>/config.json'   # ← uncomment and edit for local run

ws = Workspace.from_config(path=path)
dataset = Dataset.get_by_name(ws, name='creditcard_fraud')
df = dataset.to_pandas_dataframe()
df.head()
```

Step 3: Preprocess — Standardise `Amount`

The V1–V28 features are already PCA-transformed and unit-normalised by the dataset authors. `Amount` is the only raw feature and sits on a completely different scale (dollars). Standardising it to zero mean / unit variance puts it on the same footing as the PCA

components, preventing the model from over-weighting large transactions simply because their magnitude is large.

`Time` is dropped because it encodes wall-clock seconds elapsed since the first transaction in the batch — not a meaningful signal for cross-session fraud detection.

In [ ]:
```
# Z-score normalise Amount so it is on the same scale as the PCA features V1–V28
df['Amount'] = (df['Amount'] - df['Amount'].mean()) / df['Amount'].std()

# Time is an artifact of dataset capture order, not a fraud signal — drop it
X = df.drop(columns=['Class', 'Time'])   # feature matrix
y = df['Class']                           # ground-truth labels (0=normal, 1=fraud)
```

Step 4: Train — Isolation Forest

Isolation Forest detects anomalies by building an ensemble of random trees and measuring how many splits are needed to isolate each sample. Anomalies — rare, structurally different records — require fewer splits than normal points and therefore receive shorter average path lengths, which translate into higher anomaly scores.

The `contamination` Parameter Is a Business Decision

`contamination` is the single most consequential hyperparameter in this pipeline from a business perspective. It sets the decision threshold that converts continuous anomaly scores into binary fraud/not-fraud labels.

```
Lower contamination (e.g. 0.0005)          Higher contamination
(e.g. 0.005)
———————————————————————————————
———————————————————————————————
Fewer transactions flagged                 More transactions
flagged
↑ Precision (flags are reliable)           ↓ Precision (more false
positives)
↓ Recall (more fraud slips through)        ↑ Recall (catches more
real fraud)
Lower analyst review cost                  Higher analyst review
cost
Higher fraud loss exposure                 Lower fraud loss
exposure
Preferred by: Product, UX                  Preferred by: Risk,
Compliance
```

Current setting: `0.0017` matches the known fraud prevalence in this dataset (~0.17% of transactions). This is a reasonable starting point but **not** an optimised production value — it was chosen to reflect ground truth, not to maximise business utility. The optimal value depends on:

1. The ratio of analyst review cost to average fraud loss in your specific portfolio
2. Your regulatory obligations (some jurisdictions mandate minimum fraud detection rates)
3. Your customer experience SLA on false declines

> **Rule of thumb for calibration:** If your average fraud loss is €120 and each analyst review costs €20, you should be willing to accept up to 6 false positives per true positive before the economics invert. That implies a minimum required precision of ~14% — far lower than the current 29%. This model is **over-flagging relative to what the economics require**, which means `contamination` could be *reduced* to improve customer experience without materially increasing net fraud losses.

In [ ]:
```
model = IsolationForest(contamination=0.0017, random_state=42)
model.fit(X)

# IsolationForest.predict() returns: -1 (anomaly) or +1 (normal)
# Remap to the dataset convention: 1=fraud, 0=normal
y_pred = model.predict(X)
y_pred = [1 if x == -1 else 0 for x in y_pred]
```

Step 5: Evaluate

`classification_report` computes per-class metrics against the ground-truth `Class` label.

Metric	Definition
Precision	Of all transactions flagged as fraud, what fraction were genuinely fraudulent
Recall	Of all actual fraud cases, what fraction did the model surface
F1-score	Harmonic mean of precision and recall
Support	Sample count per class in the ground-truth labels

Expected results on this dataset:

Class	Description	Precision	Recall	F1	Support
0	Normal	~1.00	~1.00	~1.00	284,315
1	Fraud	~0.29	~0.28	~0.28	492

Cost-Benefit Analysis of Model Performance

Overall accuracy reads ~99.9%, but this is structurally misleading on an imbalanced dataset — a classifier that labels every transaction as normal achieves the same accuracy score. The

two metrics that actually drive business outcomes are **precision** (false positive rate) and **recall** (false negative rate).

Translating metrics into money

Using conservative industry-average figures (adjust these to your actual portfolio data before presenting to Finance):

Outcome	Count (this dataset)	Unit cost	Total cost
True Positives (fraud correctly caught)	~138 of 492 fraud cases (28% recall)	—€120 prevented loss each	**−€16,560** (savings)
False Negatives (fraud missed)	~354 of 492 fraud cases	+€120 loss each	**+€42,480** (unrecovered losses)
False Positives (legitimate tx flagged)	~338 flags that were not fraud (71% of all flags)	+€20 review cost each	**+€6,760** (operational overhead)
True Negatives (legitimate tx passed)	~283,977	€0	€0

Net cost of running this model on this dataset: approximately +€32,680 in unrecovered losses + review overhead.
Cost of running *no* model (accepting all fraud): +€59,040.
Model value: ~€26,360 saved — but leaves 72% of fraud losses on the table.

> This analysis uses the Kaggle dataset as a proxy. In production, substitute your portfolio's actual average transaction value, fraud rate, chargeback fee structure, and analyst hourly cost to get accurate figures. A 1% improvement in recall on a portfolio processing €1B/year at 0.17% fraud prevalence is worth approximately **€170,000 in recovered losses**.

Breakeven sensitivity

The model generates net savings as long as:

```
(recall × avg_fraud_loss) > (FP_rate × review_cost)
```

At current performance (28% recall, 71% FP rate among flags), this holds only because fraud losses significantly exceed review costs. If analyst costs rise or fraud prevalence falls, the breakeven shifts — making precision improvements more urgent than recall improvements.

Risk Assessment

Risk	Likelihood	Impact	Mitigation
Concept drift — fraud patterns shift over time; model trained on	High	High	Schedule monthly retraining; monitor precision/recall in

Risk	Likelihood	Impact	Mitigation
2013 data will degrade on current transactions			production via Azure ML monitoring
Threshold miscalibration — `contamination` set to dataset prevalence, not business-optimal value	Medium	Medium	Run threshold sweep (see Improvement Roadmap); A/B test against current rules engine
Feature leakage — V1–V28 are PCA-transformed; the original features are not disclosed, limiting interpretability for regulators	Low	High	Document the PCA provenance; confirm with data provider whether raw features are available under NDA
Training on full dataset — model is fit and evaluated on the same data (no held-out test set), inflating apparent performance	High	Medium	Implement a proper train/validation/test split before production; see Improvement Roadmap
Single-model brittleness — adversarial fraudsters can probe and adapt to a static model	Medium	High	Ensemble with rule-based signals; retrain frequently; consider online learning approaches
Regulatory non-compliance — automated adverse action without explainability may breach PSD2 Art. 98, GDPR Art. 22	Low–Medium	Very High	SHAP outputs (Step 7c) provide per-decision feature attribution; document and retain these for audit

In [ ]:
```
print(classification_report(y, y_pred))
```

Step 6 (Optional): Serialise and Register the Model

`joblib.dump` serialises the fitted `IsolationForest` object to a `.pkl` file. `Model.register` pushes it into **Assets** → **Models** in your workspace, creating a versioned, auditable artefact that can be referenced by downstream deployment pipelines (see `AzureAIMLStudioAssetsBatchEndpoint` in `ml_studio.py`).

Deployment Recommendations

Do not deploy this model as a hard transaction block. At 29% precision, blocking every flagged transaction would decline approximately 3 legitimate purchases for every fraud case stopped. The recommended architecture is a **three-tier risk scoring system**:

```
Anomaly Score (continuous)
         │
         ▼
┌──────────────────────┬──────────────────────────┬──────────────────────────
│  LOW RISK            │  MEDIUM RISK             │  HIGH RISK
│
│  Score < 0.3         │  Score 0.3 – 0.7         │  Score > 0.7
│
```

```
│                    │                          │
│
│  Auto-approve      │  Step-up authentication  │  Auto-decline +
SMS OTP  │
│  No friction       │  (OTP, biometric prompt) │  Flag for analyst
review │
│                    │                          │
│
│  ~99.5% of tx      │  ~0.3% of tx             │  ~0.2% of tx
│
└────────────────────┴──────────────────────────┴──────────────────────────
```

Threshold boundaries are illustrative — calibrate against your portfolio's ROC curve using a held-out validation set before go-live.

Pre-production checklist

- [ ] Retrain on a proper train/test split (current notebook trains and evaluates on the same data)
- [ ] Run a contamination sweep (0.0005 to 0.005) and plot the precision-recall curve to identify the business-optimal threshold
- [ ] Shadow-mode deployment: run model in parallel with existing rules engine for 4–6 weeks before routing live decisions through it
- [ ] Define SLOs: maximum acceptable false positive rate, minimum recall, maximum decision latency (target <100ms for real-time scoring)
- [ ] Configure Azure ML monitoring for data drift detection on V1–V28 feature distributions
- [ ] Implement SHAP logging at inference time to satisfy explainability requirements for adverse-action decisions
- [ ] Set a retraining trigger: alert when rolling 30-day precision drops >5 percentage points from baseline

In [ ]:
```python
import joblib

joblib.dump(model, 'isolation_forest.pkl')

Model.register(
    model_path='isolation_forest.pkl',
    model_name='creditcard_if_model',
    workspace=ws,
)
```

Step 7: Visualise Results

> **Stakeholder communication note:** The three visualisations below serve different audiences. Present **7a** to Risk and Finance as a headline summary of

> model volume. Present **7b** to Product to demonstrate that most flagged transactions are not simply large-value outliers — the model is responding to behavioural patterns, not just transaction size. Present **7c** to Compliance as evidence that the model's decisions are explainable and auditable at the feature level.

7a — Predicted Class Distribution

A count plot of the model's binary output. Given `contamination=0.0017`, expect roughly 480–500 transactions flagged as fraud (1) out of ~284,807 total — proportional to the contamination rate.

How to read this for stakeholders: The bar heights are a direct consequence of the `contamination` threshold setting, not an independent validation of accuracy. A much taller fraud bar than expected indicates the threshold is too permissive and analyst review queues will be overloaded; a fraud bar near zero indicates the threshold is too conservative and fraud is passing through unchecked.

In [ ]:
```
import matplotlib.pyplot as plt
import seaborn as sns

df['predicted_anomaly'] = y_pred

sns.countplot(x='predicted_anomaly', data=df)
plt.title('Predicted Class Distribution (0=Normal, 1=Fraud)')
plt.xlabel('Predicted label')
plt.ylabel('Transaction count')
plt.show()
```

7b — Transaction Amount by Predicted Class

Compares the standardised `Amount` distribution across the model's two output classes. Because Isolation Forest operates on all 29 features simultaneously, this plot isolates whether transaction size alone is driving the fraud flags — which would suggest the model has learned a proxy rule rather than genuine fraud patterns.

Interpreting the result for Product and Risk stakeholders:

- If fraud-flagged transactions (`1`) show a markedly wider IQR or more high-value outliers than normal transactions (`0`): the model is partially proxying transaction size as a fraud signal. This is plausible but incomplete — small-value card testing fraud (common in carding attacks) would be missed. Inform Risk that the model has a systematic blind spot for low-value fraud.
- If the distributions are nearly identical: the model is responding primarily to patterns in the PCA-derived V-features (behavioural/device signals), which is the more robust signal. This is the preferred outcome — communicate to Product that declined

transactions are not simply "large purchases" and that adding high-value purchase friction (e.g. 3DS prompts on transactions over €500) would not have the same effect as the model.

In [ ]:
```
plt.figure(figsize=(10, 6))
sns.boxplot(data=df, x='predicted_anomaly', y='Amount')
plt.title('Transaction Amount (standardised) by Predicted Class')
plt.xlabel('Predicted label (0=Normal, 1=Fraud)')
plt.ylabel('Amount (z-score)')
plt.show()
```

7c — SHAP Beeswarm: Feature Importance

SHAP (SHapley Additive exPlanations) attributes each prediction to its contributing features by computing Shapley values from cooperative game theory. Each dot is one transaction; each row is one feature; horizontal position encodes impact magnitude and direction; colour encodes the feature's raw value (red=high, blue=low).

Reading the plot:

- Features are sorted by mean absolute SHAP value — the topmost rows are the strongest drivers of the model's fraud decisions.
- A cluster of red dots far right on a feature means high values of that feature push the model toward anomaly.
- A tight cluster near zero for a feature means it contributes little to the decision for these samples.

Performance note: SHAP on tree ensembles via `TreeExplainer` is exact and fast. The `Explainer` call here auto-selects `TreeExplainer`. The slice `X[:100]` is used only for the beeswarm plot — running the full 284k rows is feasible but slower.

Regulatory and Compliance Use of This Output

Under **PSD2 Article 98** (Strong Customer Authentication) and **GDPR Article 22** (automated decision-making), payment systems that take automated adverse action against a customer — declining a transaction, triggering additional authentication — may be required to provide a human-reviewable explanation.

SHAP satisfies this requirement at the individual transaction level: for each flagged transaction, you can identify the top 3–5 features that drove the anomaly score and surface these to a fraud analyst in the review queue. **This notebook should be treated as the explainability baseline that Compliance signs off on before any production deployment.**

What to communicate to Compliance:

- The top features identified by SHAP are PCA-transformed components (V1–V28), not raw customer attributes — the original feature identities are not disclosed in this public dataset. Confirm with your data provider whether raw feature names can be mapped back before presenting individual SHAP explanations to customers.
- SHAP values are additive and locally faithful — each value is the exact marginal contribution of that feature to the anomaly score for that specific transaction. This is a stronger guarantee than gradient-based or surrogate-model explanations.
- Log SHAP values at inference time and retain them for the duration required by your data retention policy.

In [ ]:
```
import shap

# shap.Explainer auto-selects TreeExplainer for IsolationForest — exact, not approx
explainer = shap.Explainer(model, X)
shap_values = explainer(X[:100])    # sample 100 rows for a readable beeswarm; incr
shap.plots.beeswarm(shap_values)
```

Improvement Roadmap and Stakeholder Communication Plan

This section consolidates the technical improvement recommendations and the communication strategy for model limitations into a single reference for engineering leads, product managers, and the risk committee.

Model Improvement Roadmap

The following improvements are prioritised by expected impact-to-effort ratio. Items in **Phase 1** should be completed before any production deployment. Items in **Phase 2** represent the target steady-state for a production fraud system.

Phase 1 — Pre-deployment (Required)

Priority	Improvement	Technical approach	Expected impact
🔴 Critical	Fix evaluation methodology	Implement proper train/validation/test split (e.g. 60/20/20); current notebook evaluates on training data, inflating all metrics	Reported metrics will change — likely downward; establishes honest baseline
🔴 Critical	Threshold optimisation	Sweep `contamination` from 0.0005 to 0.005; plot precision-recall curve; select operating point based on cost-benefit ratio from Step 5 analysis	Directly improves the FP/FN trade-off; expected +5–15pp precision with minimal recall loss

Priority	Improvement	Technical approach	Expected impact
High	Add a supervised model for comparison	Train an XGBoost or LightGBM classifier on the labelled data; use IsolationForest predictions as a feature or as an ensemble member	Labelled data exists — supervised models typically achieve F1 >0.80 on this dataset
High	Handle class imbalance explicitly	Apply SMOTE oversampling or `class_weight='balanced'` in a supervised model; or use `IsolationForest` score as a soft feature rather than a hard predictor	Directly addresses the 492 vs 284,315 imbalance that drives the low fraud-class recall

Phase 2 — Production Hardening (Target State)

Priority	Improvement	Technical approach	Expected impact
Medium	Scheduled retraining pipeline	Implement Azure ML Pipelines with a monthly retraining trigger; monitor V1–V28 distribution drift using Azure ML data drift detector	Prevents model decay as fraud patterns evolve
Medium	Real-time scoring endpoint	Convert `BatchEndpoint` to a real-time `ManagedOnlineEndpoint` for sub-100ms inference; required for card-present transaction flows	Enables real-time blocking rather than post-hoc review
Medium	Feature engineering	Derive velocity features: number of transactions in last 1h/24h per card; average spend deviation; merchant category risk score	Velocity features are among the strongest fraud signals in production systems
Low	Online learning / continual training	Incorporate confirmed fraud labels from the dispute resolution process as they arrive; use partial_fit or periodic retraining on recent windows	Closes the feedback loop between model predictions and ground truth
Low	Ensemble with rules engine	Combine model score with a rules engine (velocity caps, country mismatch, device fingerprint) using a logistic meta-learner	Rules catch known fraud patterns instantly; model catches novel patterns

> **Tool recommendation:** Before implementing Phase 2, run this notebook through an AI code review (Claude, GitHub Copilot, or similar) with the prompt: "*Review this Isolation Forest fraud detection notebook and identify: (1) data leakage risks, (2) production-readiness gaps, (3) feature engineering opportunities, and (4) monitoring requirements.*" AI-assisted code review on a notebook of this size typically surfaces 5–10 actionable issues in under 2 minutes.

Risk Mitigation Strategies

Risk	Mitigation strategy	Owner	Review cadence
Model decay / concept drift	Deploy Azure ML data drift monitor on V-feature distributions; set alert threshold at >10% PSI on any top-5 SHAP feature; trigger emergency retraining	MLOps	Weekly automated check
Regulatory adverse action without explanation	Log SHAP feature attributions at inference time; expose top-3 drivers in analyst review UI; retain logs per data retention policy	Engineering + Compliance	At deployment; audit annually
Threshold miscalibration	A/B test new threshold against current system on 5% of traffic in shadow mode for 30 days before full rollout; require Risk sign-off on contamination changes	Risk + MLOps	Before each retraining cycle
Training data leakage	Enforce temporal split: train on transactions before a cutoff date, test on transactions after; never use future labels to train past predictions	Engineering	Code review gate
Single point of failure	Do not route 100% of decisions through the ML model alone; maintain a parallel rules-based fallback; define circuit-breaker logic for model endpoint failures	Engineering	Incident response plan
Adversarial adaptation	Rotate model versions on an irregular schedule; do not publish model scores or thresholds publicly; monitor for score distribution shifts that suggest probing	Security + MLOps	Quarterly

---

Stakeholder Communication Plan

Guiding principle

Stakeholders should always receive metrics translated into business outcomes, never raw ML metrics in isolation. "Precision of 0.29" means nothing to a CFO; "for every genuine fraud case we stop, we generate 3 unnecessary customer service calls" is immediately actionable.

Communication by audience

Risk Committee / Board (quarterly)

Report: Total fraud prevented (€), false positive volume, net programme cost vs. savings, regulatory compliance status.

Template: "*The model prevented €X in fraud losses this quarter by flagging Y transactions for review. Of those, Z% were confirmed fraud. The remaining W% were false positives, each generating an average of €20 in review cost. Net benefit: €X − (W × €20) = €[net]. The model currently catches [recall%] of all fraud — the remaining [1-recall%] results in chargeback liability. Our Phase 2 improvements target lifting recall to [target]% by [date].*"

Product and UX (monthly)

Report: False positive rate on transactions by customer segment, friction events, conversion impact.

Template: "*The model flagged [N] legitimate transactions this month, affecting [N] customers who experienced step-up authentication or declines. Of these, [X%] completed the transaction after step-up; [Y%] abandoned. Estimated lost GMV: €[Z]. The model is currently weighted toward [precision/recall] — adjusting the threshold by [amount] would reduce friction events by approximately [N] at the cost of [M] additional missed fraud cases worth €[€].*"

Engineering / MLOps (weekly)

Report: Endpoint latency P50/P99, model version in production, drift alert status, retraining queue.

Template: Standard Azure ML monitoring dashboard; alert on P99 latency >200ms, any data drift alert firing, precision rolling 30-day drop >5pp.

Customer Service (as needed — fraud dispute spikes)

Alert trigger: When false positive volume exceeds rolling 7-day average by >2σ, notify CS leadership with: estimated volume of affected customers, recommended communication script, ETA on threshold review.

What to communicate about model limitations — always, to every audience

1. **This model is a decision-support tool, not an autonomous decision-maker.** All auto-decline or auto-block actions should have a human review pathway.
2. **Current fraud detection rate is 28%.** The model is designed to be one layer in a defence-in-depth system, not the sole fraud control.
3. **The model was trained on 2013 European card transaction data.** Fraud patterns have evolved significantly. Performance on current transaction data will differ and should be validated on a representative recent sample before deployment.
4. **Model decisions are explainable.** SHAP feature attributions are logged per decision and available for regulatory audit or customer dispute response.